Mean-Variance Optimization Examples

Efficient Frontiers, Portfolio Strategies, and Practical Solutions

IEOR E4576 — Data-Driven Models in Finance — Fall 2026 — Week 8

Based on *Machine Learning for Algorithmic Trading*, Stefan Jansen — Chapter 5, pp. 132–172

1 The Optimization Problem

Given N assets with expected return vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$, we want to find the portfolio weights $\mathbf{w} \in \mathbb{R}^N$ that optimally trade off return against risk. The standard formulation in practice is the **risk-return trade-off**:

$$\begin{aligned} \max_{\mathbf{w}} \quad & \mathbf{w}^T\boldsymbol{\mu} - \lambda \mathbf{w}^T\boldsymbol{\Sigma}\mathbf{w} \\ \text{subject to} \quad & \mathbf{w}^T\mathbf{1} = 1 \\ & w_i \geq 0 \text{ for } i = 1, \dots, N \end{aligned} \tag{1}$$

where $\lambda \geq 0$ is the risk aversion parameter. The budget constraint $\mathbf{w}^T\mathbf{1} = 1$ applies to long-only portfolios; a long-short portfolio uses $\mathbf{w}^T\mathbf{1} = 0$ instead.

The objective is concave and the constraints are linear, so (1) is a convex quadratic program. Position limits ($w_i \leq w^{\max}$) keep it a convex QP.

The risk aversion λ moves the solution along the efficient frontier:

- $\lambda \to 0$: risk is ignored, and the optimizer puts everything in the asset with the highest expected return.
- $\lambda \to \infty$: return is ignored, and the solution is the **minimum variance portfolio**.

Core idea. MVO maximizes expected return minus a penalty on variance. One parameter, λ, sets the trade-off. Solving (1) for a range of λ traces the **efficient frontier**: the set of optimal risk-return trade-offs.

2 Implementation with `scipy.optimize`

2.1 The MVO Solver

`scipy.optimize.minimize` minimizes, so we pass the negative of the objective in (1).

```
import numpy as np
from scipy.optimize import minimize

# Inputs: mu (N,), cov (N, N)
def mvo(mu, cov, lam):
    N = len(mu)
    result = minimize(lambda w: -(w @ mu - lam * w @ cov @ w),
                      np.ones(N) / N,              # equal-weight start
                      method='SLSQP',
                      bounds=[(0, 1)] * N,         # long-only
                      constraints=[{'type': 'eq',
                                    'fun': lambda w: w.sum() - 1}])
    return result.x

w_mvo = mvo(mu, cov, lam=5)
```

2.2 Minimum Variance Portfolio

The minimum variance portfolio is the limit $\lambda \to \infty$. A very large λ gives it with the same solver:

```
w_minvar = mvo(mu, cov, lam=1e4)
```

2.3 Efficient Frontier

To trace the efficient frontier, solve (1) on a grid of risk aversions. A logarithmic grid covers the range from aggressive to conservative portfolios:

```
lambdas = np.logspace(-1, 3, 30)
frontier = [mvo(mu, cov, lam) for lam in lambdas]

frontier_rets = np.array([w @ mu for w in frontier])
frontier_vols = np.array([np.sqrt(w @ cov @ w) for w in frontier])
```

2.4 Maximum Sharpe Ratio Portfolio

The maximum Sharpe ratio portfolio is one point on the frontier. Pick the λ on the grid with the highest ratio of return to volatility:

```
sharpes = frontier_rets / frontier_vols
w_maxsharpe = frontier[np.argmax(sharpes)]
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 6))
plt.plot(frontier_vols, frontier_rets, 'b-', lw=2,
         label='Efficient Frontier')
plt.scatter(np.sqrt(w_minvar @ cov @ w_minvar), w_minvar @ mu,
            c='red', s=100, zorder=5, label='Min Variance')
plt.scatter(np.sqrt(w_maxsharpe @ cov @ w_maxsharpe),
            w_maxsharpe @ mu, c='green', s=100,
            marker='*', zorder=5, label='Max Sharpe')
plt.xlabel('Annualised Volatility')
plt.ylabel('Annualised Return')
plt.title('Efficient Frontier')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

3 Alternative Portfolio Strategies

3.1 Equal Weight

The simplest allocation: $w_i = 1/N$ for all i.

```
w_equal = np.ones(N) / N
ret_equal = w_equal @ mu
vol_equal = np.sqrt(w_equal @ cov @ w_equal)
sharpe_equal = ret_equal / vol_equal
```

Equal weight is a surprisingly hard benchmark to beat. DeMiguel et al. (2009) showed that $1/N$ outperforms many MVO variants out of sample, especially when estimation error is large.

3.2 Risk Parity (Inverse Volatility)

Weight each asset inversely proportional to its volatility:

$$w_i = \frac{1/\sigma_i}{\sum_{j=1}^{N} 1/\sigma_j}$$

```
vols = np.sqrt(np.diag(cov))
w_riskparity = (1 / vols) / np.sum(1 / vols)
ret_rp = w_riskparity @ mu
vol_rp = np.sqrt(w_riskparity @ cov @ w_riskparity)
```

Risk parity does not require expected return estimates, making it robust to return estimation error. It tends to produce well-diversified portfolios with moderate turnover.

4 Comparison of Portfolio Strategies

Strategy	Requires $\boldsymbol{\mu}$?	Key Advantage	Key Weakness
MVO (λ)	Yes	Uses return forecasts and risk together	Very sensitive to $\hat{\boldsymbol{\mu}}$ errors
Min Variance ($\lambda \to \infty$)	No	Only needs $\boldsymbol{\Sigma}$	Ignores expected returns entirely
Equal Weight	No	Zero estimation error	No risk adjustment
Risk Parity	No (only σ_i)	Well diversified, stable	Ignores correlations and returns

5 Complete Working Example

```python
import numpy as np
from scipy.optimize import minimize
from sklearn.covariance import LedoitWolf

np.random.seed(42)

# --- 1. Simulate 30 stocks, 500 days ---
N, T = 30, 500
market = np.random.normal(0.0004, 0.012, T)
returns = np.column_stack([
    np.random.uniform(0.5, 1.5) * market
    + np.random.normal(0, np.random.uniform(0.008, 0.02), T)
    for _ in range(N)
])

mu = returns.mean(axis=0) * 252
cov_lw = LedoitWolf().fit(returns).covariance_ * 252

# --- 2. Risk-return trade-off MVO ---
def mvo(mu, cov, lam):
    N = len(mu)
    result = minimize(lambda w: -(w @ mu - lam * w @ cov @ w),
                      np.ones(N) / N, method='SLSQP',
                      bounds=[(0, 1)] * N,
                      constraints=[{'type': 'eq',
                                    'fun': lambda w: w.sum() - 1}])
    return result.x

def portfolio_stats(w, mu, cov):
    ret = w @ mu
    vol = np.sqrt(w @ cov @ w)
    return ret, vol, ret / vol

# --- 3. Compute portfolios ---
lambdas = np.logspace(-1, 3, 30)
frontier = [mvo(mu, cov_lw, lam) for lam in lambdas]
sharpes = [portfolio_stats(w, mu, cov_lw)[2] for w in frontier]

w_mvo = mvo(mu, cov_lw, lam=5)                  # moderate risk aversion
w_mv = mvo(mu, cov_lw, lam=1e4)                 # lambda -> infinity: min variance
w_ms = frontier[int(np.argmax(sharpes))]        # best Sharpe on the lambda grid
w_eq = np.ones(N) / N
vols = np.sqrt(np.diag(cov_lw))
w_iv = (1 / vols) / np.sum(1 / vols)

# --- 4. Compare ---
strategies = {'MVO (lambda=5)': w_mvo, 'Max Sharpe': w_ms,
              'Min Variance': w_mv, 'Equal Weight': w_eq,
              'Inverse Vol': w_iv}
for name, w in strategies.items():
    r, v, s = portfolio_stats(w, mu, cov_lw)
    print(f"{name:15s}  Ret={r:.2%}  Vol={v:.2%}  Sharpe={s:.2f}  "
          f"Holdings={np.sum(w > 1e-4)}")
```

Output:

```
MVO (lambda=5)   Ret=48.93%  Vol=21.15%  Sharpe=2.31  Holdings=5
Max Sharpe       Ret=45.87%  Vol=19.81%  Sharpe=2.32  Holdings=6
Min Variance     Ret=4.89%  Vol=12.00%  Sharpe=0.41  Holdings=13
Equal Weight     Ret=12.18%  Vol=16.68%  Sharpe=0.73  Holdings=30
Inverse Vol      Ret=10.95%  Vol=15.93%  Sharpe=0.69  Holdings=30
```

These are in-sample numbers: the same returns are used to estimate $\boldsymbol{\mu}$ and to evaluate the portfolios, so the MVO Sharpe ratios are far too optimistic. Note also how concentrated the MVO portfolios are, with only 5 or 6 holdings out of 30.

Further Reading:

- Markowitz, H. (1952). Portfolio selection. *Journal of Finance*, 7(1), 77–91.
- Ledoit, O. and Wolf, M. (2004). A well-conditioned estimator for large-dimensional covariance matrices. *Journal of Multivariate Analysis*, 88(2), 365–411.

- DeMiguel, V., Garlappi, L., and Uppal, R. (2009). Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy? *Review of Financial Studies*, 22(5), 1915–1953.
- Jansen, S. (2020). *Machine Learning for Algorithmic Trading*, 2nd ed., Chapter 5.